# Quarry: Technical Architecture Specification

Local Semantic Search for AI Agents

July 2026

## Contents

# 1 Overview

Quarry is a local semantic search engine. It ingests documents in 20+ formats, embeds them with a local ONNX model, stores vectors in LanceDB, and serves semantic search to AI agents (Claude Code, Claude Desktop) and a macOS menu bar app.

The system is **daemon-first** (DES-031 v2.2, ACCEPTED, in implementation): a single `quarryd` process holds the engine, and the CLI, MCP, and library are clients that reach it over a versioned wire protocol — the daemon is the intermediary service layer. As of v2-3a the engine runs only in `quarryd` (the `quarry serve` subcommand is retired), loopback requests are authenticated with a mode-0600 `serve.token`, and the CLI remote paths resolve that token through `ClientConfig`. The full client tier (`QuarryClient`) and `quarry mcp`-as-client are in flight (v2-3/v2-4); until they land, the local-mode CLI and `quarry mcp` still load the engine in-process.

## 1.1 Design Principles

**P1**. **Daemon-first.** The `quarryd` process is the single engine (embedding model, LanceDB, pipeline, sync registry); the CLI, MCP, and library are clients over a versioned REST contract (DES-031 v2.2, ACCEPTED, in implementation). The prior *library-first* principle — the core library did all work and the surfaces were thin wrappers — described the as-built state DES-031 v2 replaces.

**P2**. **Local by default.** Everything runs on the user's machine. No API keys, no cloud accounts. The embedding model downloads once (∼120 MB, int8 quantized).

**P3**. **One daemon, many clients.** A single `quarryd` process loads the embedding model once and serves all sessions over the REST wire; supervised by launchd `KeepAlive` / systemd `Restart=always`.

**P4**. **Fire-and-forget I/O.** Side-effect MCP tools return immediately and process in a bounded thread pool. Search is synchronous.

**P5**. **Named databases.** Each database is fully isolated: its own LanceDB directory, sync registry, and vector index. Work/personal separation without configuration files.

# 2 System Architecture

## 2.1 Deployment Topology

**Single machine (localhost TLS):**

```
                  stdio wss:// (TLS)
Claude Code <------------------> mcp-proxy <--------------------> quarry serve
           MCP JSON-RPC (~5 MB Go) pinned CA cert (one daemon)
```

**Remote server (GPU):**

```
Mac (client) Linux (server)
                  stdio wss:// (TLS)
Claude Code <------------------> mcp-proxy <----------------------> quarry serve
           MCP JSON-RPC (~5 MB Go) pinned CA cert (GPU daemon)

CLI -----> HTTPS GET/POST ---------> quarry serve
          Bearer + pinned CA
```

Without the proxy, every Claude Code tab spawns a separate Python process, each loading the embedding model into ∼200 MB of RAM. With it, startup is instant and state is shared across all sessions. Installed services use TLS with TOFU certificate pinning (DES-020) — even on localhost.

When a remote server is configured (DES-021), the CLI routes all data commands over HTTPS to the server, with only authentication and administration commands remaining local. `find`, `show`, `status`, `ingest`, `remember`, `delete`, `sync`, all `list` subcommands, and directory `register`/`deregister` route remotely. `use` is local by design: it sets the client's persistent default database, but the remote server is fixed to the database it was started with, so switching databases is a client-side setting rather than a remote call.

### 2.2 Surface Summary

| Surface | Transport | Purpose |
|---|---|---|
| CLI | Direct library calls | Interactive use from terminal |
| MCP server | stdio or WebSocket | Claude Code/Desktop tools |
| HTTP server | REST JSON + WS `/mcp` | Menu bar app + mcp-proxy |
| Plugin | Hook shell commands | Auto-register, auto-ingest, compaction safety |

## 3 Module Responsibilities

### 3.1 Data Layer (`src/quarry/db/`)

The database facade composes five subsystems behind a single entry point (`Database`), so callers pass one object rather than a raw LanceDB connection.

| Module | File | Responsibility |
|---|---|---|
| Facade | `src/quarry/db/facade.py` | `Database` — single entry point composing the store, search, catalog, schema, and optimizer subsystems (DES-031). |
| Chunk store | `src/quarry/db/chunk_store.py` | Insert, progressive-flush, delete, and count operations on the `chunks` table. |
| Chunk search | `src/quarry/db/chunk_search.py` | Vector-similarity search under the cosine metric. |
| Chunk catalog | `src/quarry/db/chunk_catalog.py` | List documents, list collections, and fetch page text. |
| Optimizer | `src/quarry/db/optimizer.py` | Table compaction, FTS index rebuild, and per-collection indexing. |
| Schema | `src/quarry/db/schema.py` | Chunks-table schema and idempotent migration (agent-memory columns, FTS index). |
| Storage | `src/quarry/db/storage.py` | LanceDB connection, database discovery, and size formatting. |

Supporting types: `src/quarry/models.py` (immutable `PageContent`, `Chunk`, `PageAnalysis` dataclasses), `src/quarry/types.py` (structural `Protocol`s for LanceDB, OCR, and embedding backends), `src/quarry/results.py` (`SearchResult`, `SearchFilter`, `WORST_CASE_DISTANCE`), and `src/quarry/config.py` (`Settings`, `resolve_db_paths()`, ONNX model constants).

### 3.2 Ingestion Pipeline (`src/quarry/ingestion/`)

| Module | File | Responsibility |
|---|---|---|
| Pipeline | `src/quarry/ingestion/pipeline.py` | Orchestrator: dispatch by format, chunk, embed, store. Entry points `ingest_document`, `ingest_content`, `ingest_url`. |
| Chunker | `src/quarry/ingestion/chunker.py` | Split page text into overlapping chunks; respects page and heading boundaries. |
| URL ingester | `src/quarry/ingestion/url_ingester.py` | Fetch, parse HTML, chunk, embed, store. `ingest_url` takes an opt-in `content_scrubber` (see PII redaction). |
| Web fetch | `src/quarry/ingestion/web_fetch.py` | Fetch a URL over HTTP(S) and return validated HTML text. |
| Ingest stats | `src/quarry/ingestion/ingest_stats.py` | Per-format ingest counters merged into an ingest result. |

Format extraction lives in `src/quarry/extractors/`; embedding and OCR provider selection in `src/quarry/ingestion/provider.py`, `src/quarry/ingestion/backends.py`, and `src/quarry/ingestion/ocr_local.`

### 3.3 Retrieval Seam (`src/quarry/retrieval/`)

Hybrid search runs through one parameterizable seam shared by every surface and by the eval harness, so the production path and the committed baseline cannot drift.

| Module | File | Responsibility |
|---|---|---|
| Service | `src/quarry/retrieval/service.py` | `SearchService` — the single call every surface (CLI, HTTP, MCP) makes for search. Kills remote/local divergence (bug class 3). |
| Hybrid | `src/quarry/retrieval/hybrid.py` | `HybridRetriever` — vector + BM25 channels fused with RRF, behind a config. |
| Config | `src/quarry/retrieval/config.py` | `RetrievalConfig` — frozen retrieval knobs (RRF $k$, over-fetch, cosine metric, decay rate, reranker). Defaults reproduce shipped search bit-for-bit. |
| Protocols | `src/quarry/retrieval/protocols.py` | `Retriever` and `Reranker` structural interfaces (one method each). |
| Fusion | `src/quarry/retrieval/fusion.py` | `RrfFusion` — Reciprocal Rank Fusion with optional temporal decay. |
| Reranker | `src/quarry/retrieval/reranker.py` | `NullReranker` — pass-through Null Object when reranking is off. |

### 3.4 Format Processors (`src/quarry/extractors/`)

Each extractor implements a common `Protocol` (`src/quarry/extractors/protocol.py`) and converts a source format into `list[PageContent]`:

| Module | File | Formats |
|---|---|---|
| PDF | `src/quarry/extractors/pdf_extractor.py` | PDF (text via PyMuPDF, OCR fallback for image pages) |
| Image | `src/quarry/extractors/image_extractor.py` | PNG, JPG, TIFF, BMP, WebP (OCR) |
| Text | `src/quarry/extractors/text_extractor.py` | TXT, MD, LaTeX, DOCX |
| Code | `src/quarry/extractors/code_extractor.py` | 30+ languages (tree-sitter section splitting) |
| HTML | `src/quarry/extractors/html_extractor.py` | HTML (boilerplate stripping, Markdown conversion) |
| Spreadsheet | `src/quarry/extractors/spreadsheet_extractor.py` | XLSX, CSV (LaTeX tabular serialization) |
| Presentation | `src/quarry/extractors/presentation_extractor.py` | PPTX (slide-per-chunk with notes) |

### 3.5 Backends

| Module | File | Responsibility |
|---|---|---|
| Factory | `src/quarry/ingestion/backends.py` | Thread-safe factory with double-checked locking. `get_ocr_backend()` and `get_embedding_backend()` return cached singletons. |
| OCR | `src/quarry/ingestion/ocr_local.py` | Local OCR via RapidOCR (offline, no setup). |
| Embeddings | `src/quarry/embeddings.py` | Local ONNX embedding (snowflake-arctic-embed-m-v1.5, 768-dim, 512 tokens). Vectors are L2-normalized to unit length. |
| Provider | `src/quarry/ingestion/provider.py` | ONNX Runtime execution provider auto-detection (CUDA/FP16 or CPU/int8). |

### 3.6 Capture and Redaction

Session transcripts and WebFetch auto-captures are scrubbed at write time before any bytes reach a git-tracked (and soon pushed) surface (DES-036).

| Module | File | Responsibility |
|---|---|---|
| Capture writer | `src/quarry/capture.py` | `CaptureWriter` — the single choke point both `.md` producers route through. Scrubs, then atomically writes. |
| Scrubber | `src/quarry/scrub.py` | `Scrubber` — ordered redaction passes (secrets, paths, emails, hostname, profanity). Idempotent. |
| Scrub rules | `src/quarry/scrub_rules.py` | Secret-detection regex catalog applied by the scrubber. |
| Capture URL | `src/quarry/capture_url.py` | `CaptureUrl` — strips user-info/query/fragment from a captured URL's metadata form. |
| Web capture | `src/quarry/web_capture.py` | Parse a PostToolUse WebFetch payload into URL + fetched content. |
| Transcript | `src/quarry/transcript.py` | The session transcript file and its derived identifiers. |
| Backfill | `src/quarry/backfill.py` | Backfill historical session transcripts through the capture writer. |

### 3.7 Sync and Registry

| Module | File | Responsibility |
|---|---|---|
| Sync | `src/quarry/sync.py` | Directory sync: discover files, compute delta (new/changed/deleted), ingest/delete accordingly. |
| Discovery | `src/quarry/sync_discovery.py` | Walk, filter, and hash files for the sync delta. |
| Ingest | `src/quarry/sync_ingest.py` | Progressive per-collection ingest (producer/consumer with within-file resume). |
| Registry | `src/quarry/sync_registry.py` | SQLite registry of watched directories and per-file state. |

### 3.8 Surfaces

| Module | File | Responsibility |
|---|---|---|
| CLI | `src/quarry/__main__.py` | Typer CLI with rich formatting. Synchronous. All commands: resolve settings → call core → format output. |
| MCP Server | `src/quarry/mcp_server.py` | FastMCP server. Read-only tools are synchronous. Side-effect tools use fire-and-forget. |
| HTTP Server | `src/quarry/http_server.py` | Starlette ASGI app on uvicorn. REST endpoints + WebSocket `/mcp`. |
| Hooks | `src/quarry/hooks.py` | Claude Code plugin hooks: session-start (auto-register), post-web-fetch (auto-ingest), pre-compact (capture transcript). All fail-open. |
| Hook Entry | `src/quarry/_hook_entry.py` | Lightweight hook dispatcher that bypasses full CLI import chain. Dispatches via `sys.argv` with lazy imports (∼0.1 s vs ∼1.5 s for full CLI). |
| Remote | `src/quarry/remote.py` | Remote server config: mcp-proxy config read/write, TOFU CA cert fetch and storage, connection validation. |
| Remote client | `src/quarry/remote_client.py` | `RemoteClient` — authenticated HTTP client the CLI uses to route data commands to a remote daemon over pinned-CA HTTPS. |
| Daemon | `src/quarry/service.py` | `quarry serve` lifecycle via `launchd`/`systemd`. |

### 3.9 Support

| Module | File | Responsibility |
|---|---|---|
| Formatting | `src/quarry/formatting.py` | Pre-formatted plain text output using unicode box-drawing. Used by both CLI and MCP for consistent output. |
| Sitemap | `src/quarry/sitemap.py` | Sitemap discovery and URL extraction for web ingestion. |
| Doctor | `src/quarry/doctor.py` | Health checks and install flow: model availability, database access, MCP configuration, ethos extension setup. |
| Enable | `src/quarry/enable.py` | Project activation and deactivation: register directory, bootstrap ethos memory extensions, write per-project config. |
| TLS | `src/quarry/tls.py` | TLS certificate generation: self-signed EC P-256 CA and server cert with atomic file writes and TOFU-safe CA reuse. |
| Proxy | `src/quarry/proxy.py` | `mcp-proxy` binary download, SHA256 verification, and platform-specific installation. |
| Logging | `src/quarry/logging_config.py` | Structured logging configuration: rotating file handler + stderr, `QUARRY_LOG_LEVEL` override. |

## 4 Daemon Model

### 4.1 Why a Daemon

The embedding model (snowflake-arctic-embed-m-v1.5, int8 quantized ONNX) occupies ∼200 MB of RAM and takes ∼250 ms to load. MCP's stdio transport model spawns a new process per session. Without a daemon, five Claude Code tabs means five copies of the model in memory and five cold starts.

`quarryd` loads the model once and keeps it resident, respawned by launchd `KeepAlive` / systemd `Restart=always`.

### 4.2 Process Boundary and Loopback Authentication (v2.2)

`quarryd` (`src/quarry/daemon/launcher.py`) is a dedicated entry point — the sole engine process, making the client/engine boundary a hard *process* split. Only `src/quarry/daemon/` imports the engine; the `quarry serve` subcommand is deleted.

At startup `quarryd` refuses a remote-reachable bind that carries no operator key (an auto-generated token would be false security — it is unreadable by the remote clients that need it), and for a loopback bind mints a 256-bit `serve.token` when none is supplied. The daemon writes that token mode-0600 beside `serve.port` (shared `src/quarry/run_dir.py`) via an atomic `os.open(0o600)` + tmp-rename, and requires it on every request — closing the multi-user-host exposure where any local UID could reach the unauthenticated daemon on `127.0.0.1`. Loopback classification (`src/quarry/net.py`, `LoopbackPolicy`) is `ipaddress`-based: `localhost`/`::1`/`127.0.0.0/8` are loopback, `0.0.0.0` and unresolved names fail closed.

Clients resolve the target through `ClientConfig` (`src/quarry/client/config.py`): for a loopback URL the bearer is read *live* from `serve.token` (it rotates every daemon restart, so a stored token is never trusted); for a remote URL the stored bearer is kept. A missing loopback token fails closed with an actionable error rather than a bare 401. `ClientConfig` is the client tier's first piece; the full `QuarryClient` and `quarry mcp`-as-client land in v2-3/v2-4.

### 4.3 mcp-proxy Architecture

`quarry install` downloads `mcp-proxy` (SHA256-verified, platform-specific binary) and configures MCP clients to use it. The proxy:

1. Reads MCP JSON-RPC from stdin.
2. Forwards to `wss://localhost:8420/mcp` over WebSocket (TLS, pinned CA — even on localhost).
3. Relays responses back to stdout.

Each WebSocket connection gets its own asyncio Task with a `ContextVar` for database selection (`_db_name`), so `use("work")` in one session does not affect others.

### 4.4 CLI and the Daemon

Under DES-031 v2 (ACCEPTED, in implementation) the CLI is a thin client: it builds a request, calls the daemon over the wire protocol via `QuarryClient`, and renders the response — it does not load the embedding model or open LanceDB in-process. The prior design had `quarry find "margins"` load the model in-process and query LanceDB directly; that local-engine path is removed as the data commands migrate to `QuarryClient`.

## 5 MCP Tool Surface

| Tool | Key Parameters | Purpose | Exec |
|---|---|---|---|
| `find` | `query`, `limit`, `collection` | Semantic search | Sync |
| `ingest` | `source`, `overwrite`, `collection` | Ingest file or URL | Bg |
| `remember` | `content`, `document_name` | Ingest inline text | Bg |
| `list` | `kind` | List indexed data | Sync |
| `show` | `document_name`, `page_number` | Document metadata or page | Sync |
| `delete` | `name`, `kind` | Delete document or collection | Bg |
| `register_directory` | `directory`, `collection` | Register for sync | Bg |
| `deregister_directory` | `collection`, `keep_data` | Remove registration | Bg |
| `sync_all` | — | Sync all registered dirs | Bg |
| `status` | — | Database stats | Sync |
| `use` | `name` | Switch active database | Sync |

Bg = background (`ThreadPoolExecutor`).

Background tools use a bounded `ThreadPoolExecutor(max_workers=4)`. They return an optimistic response immediately. Settings and database connections are snapshotted at the tool boundary and passed into background functions — this avoids a race condition where the mutable `_db_name` ContextVar could change between the tool call and the background thread's execution. Exceptions in background threads are logged via `logger.exception()`, not raised.

# 6 HTTP Server

## 6.1 ASGI Application

Starlette app built by `build_app()` factory, served by uvicorn. REST handlers are sync functions — Starlette auto-detects this and runs them in its threadpool. The `/mcp` WebSocket endpoint is async.

Default port: 8420 (`DEFAULT_PORT` in `src/quarry/config.py`), overridable with `--port`.

## 6.2 Endpoints

| Path | Method | Purpose | Auth | Model |
|---|---|---|---|---|
| `/health` | GET | Server uptime and status | No | sync |
| `/ca.crt` | GET | CA certificate PEM (TOFU) | No | sync |
| `/search` | GET | Hybrid search | Yes | sync |
| `/show` | GET | Document metadata or page text | Yes | sync |
| `/documents` | GET | List documents | Yes | sync |
| `/documents` | DELETE | Delete document (returns 202) | Yes | async |
| `/collections` | GET | List collections | Yes | sync |
| `/collections` | DELETE | Delete collection (returns 202) | Yes | async |
| `/registrations` | GET | List directory registrations | Yes | sync |
| `/registrations` | POST | Register directory (returns 202) | Yes | async |
| `/registrations` | DELETE | Deregister directory (returns 202) | Yes | async |
| `/databases` | GET | List named databases | Yes | sync |
| `/status` | GET | Database statistics | Yes | sync |
| `/use` | POST | Switch active database | Yes | sync |
| `/remember` | POST | Ingest inline content (returns 202) | Yes | async |
| `/ingest` | POST | Ingest URL (returns 202) | Yes | async |
| `/sync` | POST | Sync registrations (returns 202) | Yes | async |
| `/tasks/{task_id}` | GET | Poll task status (unified) | Yes | sync |
| `/sync/{task_id}` | GET | Alias for `/tasks/{task_id}` | Yes | sync |
| `/ingest/{task_id}` | GET | Alias for `/tasks/{task_id}` | Yes | sync |
| `/mcp` | WS | MCP JSON-RPC over WebSocket | Yes | — |

See Appendix A for the full concurrency matrix across HTTP, CLI, and MCP surfaces.

## 6.3 Port File

After binding, the server writes the actual port to `~/.punt-labs/quarry/data/<db>/serve.port`. This supports `--port 0` (ephemeral assignment). Removed on shutdown via lifespan context.

# 7 Configuration

## 7.1 Settings

All settings are environment variables read via pydantic-settings.

| Variable | Default | Description |
|---|---|---|
| **Paths** | | |
| QUARRY_ROOT | ~/.punt-labs/quarry/data | Base directory for all databases |
| LANCEDB_PATH | <root>/default/lancedb | Vector database location |
| REGISTRY_PATH | <root>/default/registry.db | Directory sync SQLite database |
| LOG_PATH | ~/.punt-labs/quarry/logs/quarry.log | Log file (rotating, 5 MB, 3 backups; independent of QUARRY_ROOT) |
| **Chunking** | | |
| CHUNK_MAX_CHARS | 1800 | Target max characters per chunk (∼450 tokens) |
| CHUNK_OVERLAP_CHARS | 200 | Overlap between consecutive chunks |
| **Embedding** | | |
| EMBEDDING_MODEL | Snowflake/snowflake-arctic-embed-m-v1.5 | Model identifier (display only — the ONNX model is fixed) |
| EMBEDDING_DIMENSION | 768 | Vector dimension (display only — fixed at 768) |

### 7.2 Named Databases

`resolve_db_paths(settings, name)` maps a database name to a path under `QUARRY_ROOT`. Each database is fully isolated — its own LanceDB directory, sync registry, and vector index. Names containing path separators (`/` or \) or the literal traversal segments `.` or `..` are rejected; names like `foo.bar` are allowed. If `LANCEDB_PATH` was explicitly overridden, the override is preserved.

The persistent default database is stored in `~/.punt-labs/quarry/config.toml` as `[default] database = "name"`.

### 7.3 Chunking Tuning

**Increase** `CHUNK_MAX_CHARS` (3000–4000) for documents with long self-contained sections: legal contracts, academic papers, large source code functions.

**Decrease** `CHUNK_MAX_CHARS` (800–1200) for dense documents where each paragraph covers a distinct topic.

**Increase** `CHUNK_OVERLAP_CHARS` (400) when relevant information spans paragraph boundaries.

The embedding model has a 512-token context window. Chunks exceeding this are truncated during embedding, reducing quality for the tail.

## 8 Search and Retrieval

### 8.1 Embedding Model

snowflake-arctic-embed-m-v1.5 via ONNX Runtime:

- 768-dimensional vectors, 512-token context window
- Asymmetric retrieval: queries are prefixed with a search instruction, documents are not
- Output vectors are L2-normalized to unit length, so search runs under the **cosine** metric and similarity ($1 - \text{distance}$) is bounded true cosine in $[-1, 1]$ (DES-038)
- int8 quantized ONNX for CPU (∼120 MB), FP16 for CUDA (∼220 MB)
- Provider auto-detection: CUDA+FP16 when available, CPU+int8 fallback. `QUARRY_PROVIDER` env var overrides (`cpu`, `cuda`, or unset for auto). See DES-016 and `src/quarry/ingestion/provider.py`.
- Auto-downloads on first use via `huggingface_hub`

The model is fixed — there is no configuration to swap it. Changing the model invalidates all existing embeddings, requiring full re-ingestion.

## 8.2 OCR

RapidOCR handles all image-based text extraction. For semantic search, OCR accuracy matters less than expected — the embedding model is robust to minor OCR errors. A misspelled word rarely changes semantic meaning enough to affect search ranking.

## 8.3 Search Quality Tips

1. **Use natural language queries.** The model is trained on question-passage pairs. "What were Q3 revenue figures?" works better than "Q3 revenue."
2. **Use collection filtering.** `quarry find "auth logic" -c backend`.
3. **Re-ingest after tuning.** Chunk parameters only affect new ingestions. Run `quarry sync` after changes.
4. **Check chunk boundaries.** `quarry find -n 1 "query"` and examine the result.
5. **Use `--overwrite` when re-ingesting.** Without it, duplicate chunks accumulate.

## 8.4 Hybrid Search

Hybrid search is the **default for every query** — there is no vector-only path. All three surfaces (CLI, HTTP, MCP) construct a `SearchService` (`src/quarry/retrieval/service.py`) and call its one `search` method, so the surfaces cannot drift into different retrieval behavior. This closes bug class 3 (remote/local divergence), where the HTTP `/search` endpoint once ran vector-only search while the CLI ran hybrid. `SearchService` wraps a `HybridRetriever` parameterized by a `RetrievalConfig` whose defaults reproduce the shipped pipeline bit-for-bit.

**Channels:**

1. **Vector similarity** — ANN search via LanceDB under the cosine metric
2. **BM25 full-text search** — Tantivy index on the `text` column, created during schema migration

Results are fused using Reciprocal Rank Fusion (RRF):

$$\text{score}(m) = \sum_{\text{channel}} \frac{1}{60 + \text{rank}_{\text{channel}}(m)}$$

**FTS scoring.** FTS rows arrive from Tantivy without a vector distance. Rather than displaying a synthetic 1.00 (which made irrelevant keyword hits appear as perfect matches), each FTS-only row is annotated with its *true* cosine distance against the query vector; a row whose stored vector is missing or zero-length is assigned the worst-case distance (similarity $-1$) instead.

**Temporal decay** is not a second retrieval channel — it is an additional weighting term applied only to agent-scoped memories (non-empty `agent_handle`). An `agent_handle` filter therefore narrows the result set and enables decay; it does not switch retrieval modes.

$$w = e^{-\lambda \cdot h}$$

where $\lambda$ is the decay rate and $h$ is hours since ingestion. Unscoped documents (`agent_handle` = "") are exempt from decay (production leaves $\lambda = 0$).

## 8.5 Retrieval Seam and Evaluation

The `src/quarry/retrieval/` package is a deliberate seam (DES-037): the same `HybridRetriever` that serves production also serves the retrieval evaluation harness (`make eval`), so the committed baseline measures the retriever that actually ships. `RetrievalConfig` exposes the levers a bake-off varies — RRF $k$, over-fetch multiplier, distance metric, exact-vs-ANN vector search, reranker, and embedding strategy — without forking the code path. The harness is internal dev/CI tooling; it is not an end-user command. See `docs/eval-harness-design.md`.

Three columns extend the chunks table for agent-scoped memory:

| Column | Type | Default | Purpose |
|---|---|---|---|
| agent_handle | utf8 | "" | Agent identity scope |
| memory_type | utf8 | "" | fact, observation, opinion, procedure |
| summary | utf8 | "" | One-line summary |

Migration is idempotent — `_migrate_schema()` adds missing columns with empty-string defaults on every table open. Existing databases gain the columns transparently.

Identity tagging: the PreCompact hook reads the agent handle from `.punt-labs/ethos/config.yaml` and tags captured transcripts automatically.

### 8.6 Ethos Extension Integration

Quarry stores its configuration in ethos's generic extension mechanism (DES-008, DES-019):

`~/.punt-labs/ethos/identities/<handle>.ext/quarry.yaml`

The extension file contains two categories of data:

1. **Sidecar config** — `memory_collection` and `expertise_collections` are read directly by quarry via the filesystem (no ethos API call).
2. **Session context** — `session_context` is a YAML literal block scalar emitted verbatim by ethos at session start and before context compaction. It contains the agent's memory instructions (collection name, slash commands, memory types).

`quarry install` (step 7/7) scans all identity extension directories and appends `session_context` to any `quarry.yaml` that has `memory_collection` but no `session_context`. Uses raw file append to preserve existing YAML comments and formatting. Per-identity exception handling ensures one malformed file does not abort the rest.

Ownership boundary: ethos emits quarry's instructions without understanding them. Quarry writes its own instructions without importing ethos. The dependency is one-way — quarry depends on ethos for identity; ethos has zero knowledge of quarry's internals.

## 9 Logging and Exceptions

### 9.1 Logging Standard

Every module that performs I/O declares `logger = logging.getLogger(__name__)`. Pure data modules (`models.py`, `types.py`, `chunker.py`) do not.

Logging is configured once at the application entry point (CLI or MCP server). Library modules never call `logging.basicConfig`.

| Level | Use |
|---|---|
| DEBUG | Internal state: variable values, branch decisions, intermediate results. |
| INFO | Operational milestones: starting a job, completing a step, resource counts. |
| WARNING | Unexpected but recoverable: empty input, deprecated parameter usage. |
| ERROR | Failures that prevent completion but do not crash. Always with exception context. |

`CRITICAL` is not used. If the process must exit, it raises an exception.

Log messages use `%s`-style formatting (lazy evaluation), not f-strings. Each message includes enough context to trace an operation without reading code.

### 9.2 Exception Handling

The codebase uses built-in exception types. No custom exceptions unless a caller needs to distinguish failure modes programmatically.

| Exception | Raised when |
|---|---|
| `ValueError` | Invalid input: unsupported format, bad parameter. |
| `FileNotFoundError` | A file path argument does not exist. |
| `RuntimeError` | An operation reports failure (OCR engine error, ingestion failure). |
| `TimeoutError` | An external service exceeds its time budget. |

**Rules:**

1. Never swallow exceptions. Every `except` block must re-raise, or log with `logger.exception()` and raise a different exception, or (at system boundaries only) return a documented sentinel.
2. Never use bare `except:`.
3. Never catch `Exception` broadly in library code. Broad catches are permitted only at the outermost boundary (MCP tool handler, CLI command).
4. Exceptions carry context: include the values that caused the failure.
5. Use `try/finally` for cleanup, not `try/except`.
6. Do not use exceptions for flow control.

## 10 Security

### 10.1 Bearer Token Authentication

`quarry serve --api-key` (or `QUARRY_API_KEY` env var) gates all HTTP endpoints except `/health` behind `Authorization: Bearer <key>`.

- `/health` is exempt — load balancers need unauthenticated health checks.
- `OPTIONS` is exempt — CORS preflight requests carry no auth.
- `hmac.compare_digest` for token comparison — prevents timing attacks.
- Case-insensitive scheme per RFC 7235.
- Empty key = no auth — prevents accidental trivial bypass.
- No key = auth disabled — local use requires no key.

The server refuses to bind to a non-loopback address without `--api-key`.

### 10.2 WebSocket Security (CSWSH)

Origin-based cross-site WebSocket hijacking protection:

- If an `Origin` header is present, it must match the CORS allow-list.
- Browsers always send `Origin`; non-browser clients like `mcp-proxy` do not.
- Bearer auth is checked before WebSocket accept (close code 1008 on failure).

### 10.3 CORS

`CORSMiddleware` configured with GET and OPTIONS methods, Authorization and Content-Type headers. Default allowed origin: `http://localhost`. Configurable via `--cors-origin` (repeatable).

The server reflects the request's `Origin` header only when it matches the allow list, and adds `Vary: Origin` to signal cacheability varies by origin. When no `Origin` is present, no CORS headers are emitted.

## 10.4 TLS and TOFU Certificate Pinning

The installed service always runs with TLS enabled. `quarry serve` without `--tls` is supported for local development but must not be used for production. Certificate management uses TOFU (Trust On First Use) with self-signed CA pinning (DES-020).

**Certificate generation:** `quarry install` generates an EC P-256 CA and server certificate. Files are written atomically to `~/.punt-labs/quarry/tls/` with restricted permissions (0600 for keys, 0644 for CA cert). The server cert SAN includes the configured hostname (via `QUARRY_TLS_HOSTNAME`), localhost, and loopback addresses.

**TOFU login:**

1. Client fetches `/ca.crt` over HTTPS (verification disabled — bootstrap).
2. Displays SHA256 fingerprint for out-of-band confirmation.
3. Pins CA cert at `~/.punt-labs/mcp-proxy/quarry-ca.crt`.
4. All subsequent connections verify against the pinned CA only — system roots excluded.

**mcp-proxy:** The `quarry.toml` profile includes `ca_cert` pointing to the pinned CA. mcp-proxy builds a custom `tls.Config` with the pinned CA as the sole root, enforcing TLS 1.3 minimum.

## 10.5 Log Safety (CWE-532, CWE-117)

Uvicorn's access log is disabled (`access_log=False`), eliminating query string leakage at the source. The search handler logs only result count, never the raw query. WebSocket session keys are sanitized before logging: control characters stripped via `_CONTROL_CHAR_RE` and truncated to 64 characters to prevent log injection (CWE-117) and log flooding.

## 10.6 Capture PII Redaction (DES-036)

Captures — session transcripts and WebFetch auto-captures — are the input to the planned private shadow-repo sync (quarry-ow3k), so un-redacted content in a capture is a leak into a git-tracked (and soon pushed) surface. Every capture is scrubbed **at write time**, fail-closed, before any bytes reach disk.

**Scrubber (`src/quarry/scrub.py`).** A `Scrubber` applies redaction passes in a load-bearing order: *secrets* → *paths* → *emails* → *hostname* → *profanity*. Email must precede hostname so a hostname inside an email domain is subsumed by the whole-email redaction rather than half-redacted (which would leak the local part). Each pass emits a marker no pass can re-match, so scrubbing is idempotent — backfill may safely re-run.

- **Secrets** (`src/quarry/scrub_rules.py`): GitHub PATs, AWS keys, Anthropic/OpenAI keys, bearer headers, JWTs, PEM/GPG private key blocks, `KEY=value` env secrets, Slack tokens.
- **Paths**: home directories (`/Users/<user>/`, `/home/<user>/`) collapse to `~`, retaining the useful deeper structure.
- **Emails**: any RFC-shaped address → `[REDACTED:email]`. A placeholder redacts every address (completeness beats attribution), not an ethos-handle mapping.
- **Hostname**: bounded to the local machine name (`socket.gethostname()` + `.local` + short leaf), not arbitrary dotted-token detection — the latter has a catastrophic false-positive rate on identifiers like `github.com` or `quarry.db.facade`.

**Single write choke point (`src/quarry/capture.py`).** Both `.md` producers — the PreCompact hook (`src/quarry/hooks.py`) and the backfill path (`src/quarry/backfill.py`) — route through one `CaptureWriter`. Scrubbing runs to completion before an atomic temp-file rename, so a scrub failure writes no file and a partial or half-redacted capture is never left behind.

**WebFetch DB-ingest.** The WebFetch auto-capture path scrubs page text via an opt-in `content_scrubber` parameter on `ingest_url` (`src/quarry/ingestion/pipeline.py`). The parameter defaults to `None`: user-initiated `quarry ingest <url>`, sitemap crawls, and directory sync are *not* scrubbed — redaction is a captures concern, not a general-ingestion one. The URL's persisted metadata form is redacted separately by `CaptureUrl` (`src/quarry/capture_url.py`), which strips userinfo, query, and fragment (preserving IPv6 brackets) before the same text scrubber runs over the bare `scheme://host/path`.

# 11 Deployment

## 11.1 Single Machine

```
curl -fsSL https://raw.githubusercontent.com/.../install.sh | sh
```

Installs quarry, embedding model, mcp-proxy, Claude Code plugin, and registers a daemon with TLS on localhost.

## 11.2 Network Mode (Remote GPU)

```
export QUARRY_API_KEY=$(openssl rand -hex 32)
curl -fsSL https://raw.githubusercontent.com/.../install.sh | sh -s -- --network
```

Same as default, but binds daemon to `0.0.0.0:8420` instead of localhost. Requires `QUARRY_API_KEY`. Installs quarry, embedding model, generates TLS certificates, registers systemd service. Detects NVIDIA GPUs and swaps `onnxruntime` for `onnxruntime-gpu`. Prints CA fingerprint for client TOFU. Claude Code plugin installed if `claude` CLI is on PATH.

## 11.3 Client (Remote Access)

```
curl -fsSL https://raw.githubusercontent.com/.../install.sh | sh
quarry login <server> --api-key <token>
```

No special flag needed. Default install runs a local daemon on localhost. `quarry login` redirects queries to the remote server via TOFU certificate pinning and writes the mcp-proxy config.

## 11.4 Daemon (`quarry serve`)

```
QUARRY_API_KEY=<token> quarry serve --port 8420 --host 0.0.0.0 --tls
```

Non-loopback hosts require an API key (the server refuses to start without one).

Managed by `launchd` (macOS) or `systemd` (Linux) via `quarry install`. API key delivered via systemd `EnvironmentFile` (Linux) or launchd `EnvironmentVariables` (macOS) — never in process arguments.

## 11.5 Container (Fly.io)

Multi-stage Dockerfile, three stages:

1. **deps** (`python:3.13-slim`): installs `uv`, runs `uv sync --frozen --no-dev`. Two-pass for Docker layer caching (deps first, then source).
2. **model** (extends `deps`): downloads the embedding model at build time (∼120 MB, int8 quantized). Cached as a separate Docker layer.
3. **runtime** (`python:3.13-slim`): copies `/app` from deps, copies model cache from model stage. `QUARRY_ROOT=/data`, port 8080.

LanceDB data lives on a Fly persistent volume at `/data`. `QUARRY_API_KEY` is set via `fly secrets`. Cold start is ∼5 s with the baked-in model.

# 12 Performance

## 12.1 Critical Path

| Operation | Latency | Notes |
|---|---|---|
| Model load | ∼250 ms | One-time per process |
| Embedding (1 chunk) | ∼50 ms | Apple Silicon, CPU |
| LanceDB search | <10 ms | Vector similarity + filter |
| End-to-end search | <100 ms | After model is loaded |

### 12.2 Fire-and-Forget MCP Pattern

Side-effect MCP tools return immediately and process in a bounded thread pool (4 workers). This is necessary because:

1. MCP tool calls block the LLM's response stream. A 30-second ingest blocks Claude from responding.
2. The bounded pool prevents resource exhaustion under concurrent requests.
3. Settings and database connections are snapshotted at the tool boundary, avoiding race conditions with the mutable `_db_name` ContextVar.

### 12.3 Single Table Design

All chunks live in one LanceDB table (`chunks`). Document and collection boundaries are columns, not separate tables. This simplifies cross-document search and avoids table proliferation. Filtering by document/collection/page_type/source_format uses LanceDB's built-in filter predicates on the vector search.

## 13 Plugin and Hook System

The Claude Code plugin (`.claude-plugin/`) registers hooks that fire on session events and tool calls. All hooks are fail-open: exceptions are logged and the process exits 0.

| Hook Event | Matcher | Effect | Blocks? |
|---|---|---|---|
| SessionStart | * | Auto-register directory + background sync | No |
| PostToolUse | quarry tools | Suppress tool output formatting | No |
| PostToolUse | WebFetch | Auto-ingest fetched URL (scrubbed) | No |
| PreCompact | * | Capture transcript as session notes (scrubbed) | No |

### 13.1 Session Start

`handle_session_start()` auto-registers the working directory and launches a background sync via a detached subprocess. Uses atomic PID-file locking (`O_CREAT|O_EXCL`) at `~/.punt-labs/quarry/sync.pid` to prevent concurrent syncs. Returns `additionalContext` immediately without waiting.

### 13.2 Web Fetch Capture

`handle_post_web_fetch()` ingests URLs fetched during a session into the `web-captures` collection. Deduplicates by document name. Prefers already-fetched content from the tool response (avoids re-fetch, reduces SSRF surface). Both ingress branches (primary and the `ingest_url` re-fetch fallback) scrub page text and redact the URL metadata before content reaches the pushable `web-captures` collection (DES-036).

### 13.3 Pre-Compact Capture

`handle_pre_compact()` reads the transcript (JSONL) and ingests it into the `session-notes` collection. Extracts user/assistant message text, skipping tool-use blocks. Capped at 500,000 characters. The capture file is written through the shared `CaptureWriter`, which scrubs secrets, PII, and profanity before the atomic write (DES-036).

## 14 CLI Commands

| Command | Purpose |
|---|---|
| `ingest <source>` | Ingest file or URL. Flags: `--agent-handle`, `--memory-type`, `--summary` |
| `remember` | Ingest stdin content |
| `find <query>` | Semantic search |
| `show <document>` | Document metadata or page text |
| `status` | Database statistics |
| `use <name>` | Set persistent default database |
| `delete <name>` | Delete document or collection |
| `register <directory>` | Register for incremental sync |
| `deregister <collection>` | Remove directory registration |
| `sync` | Sync all registered directories |
| `enable [directory]` | Enable quarry knowledge capture for a project directory |
| `disable [directory]` | Disable quarry knowledge capture; remove registration and data |
| `optimize` | Compact LanceDB table and rebuild indexes (CLI-only). Flags: `--force` |
| `list documents` | List indexed documents |
| `list collections` | List collections with counts |
| `list databases` | List named databases |
| `list registrations` | List registered directories |
| `serve` | Start HTTP API server |
| `mcp` | Start MCP server (stdio) |
| `login <host>` | TOFU login to remote server (cert pinning) |
| `logout` | Disconnect from remote server |
| `remote list` | Show remote config; `--ping` validates connection |
| `install` | Set up data directory, download model, generate TLS certs, register daemon, detect GPU, configure ethos ext |
| `doctor` | Check environment health |
| `uninstall` | Remove system daemon |
| `version` | Print version |

Global flags: `--json`, `--verbose`/`-v`, `--quiet`/`-q`, `--db <name>`.

**Remote routing (DES-021):** When a remote server is configured (via `quarry login`), all data commands should route to the remote server over HTTPS with the pinned CA cert. Only authentication commands (`login`, `logout`, `remote`) and administration commands (`install`, `uninstall`, `serve`, `mcp`, `version`) are local-only. All data commands — `find`, `show`, `status`, `ingest`, `remember`, `delete`, `sync`, every `list` subcommand, and directory `register`/`deregister` — route remotely. `use` is local-only by design (the remote server is fixed to its own database).

# 15 Test Architecture

Roughly 1,500 tests across the suite. Three tiers plus integration:

| Tier | Strategy | Files |
|---|---|---|
| Tier 1: Core logic | Real data, no mocks | test_database, test_chunker, test_embeddings, test_code_processor, test_html_processor, test_text_processor, test_collections, test_config, test_formatting, test_image_analyzer, test_latex_utils, test_models, test_pdf_analyzer, test_presentation_processor, test_spreadsheet_processor, test_text_extractor, test_registry (17 files) |
| Tier 2: Orchestration | Mocked I/O | test_pipeline, test_pipeline_images, test_ocr_local, test_url_ingestion, test_sitemap, test_backends, test_sync, test_hooks, test_doctor (9 files) |
| Tier 3: Surface wiring | Fully mocked, <2 s | test_cli, test_mcp_server, test_mcp_websocket, test_http_server, test_proxy, test_service (6 files) |
| Integration | Real ONNX + real LanceDB | test_integration (@pytest.mark.slow) |

Quality gates: `ruff` (lint + format), `mypy` (strict), `pyright` (strict), `pytest` (all markers except slow/integration). Zero violations required.

# 16 Formal Model

The plugin state machine — how quarry hooks into the Claude Code session lifecycle — is formally specified in `docs/claude-code-quarry.tex` using Z notation (ISO 13568).

The specification covers:

- Session start with/without auto-sync
- Web fetch auto-capture into `web-captures` collection
- Pre-compaction transcript capture into `session-notes`
- Database switching via `use` tool
- Knowledge hint injection at tool boundaries
- Compaction observability: a capture flag records whether capture preceded compaction; the hook system enforces capture-before-compaction on a best-effort basis
- Clean session boundaries: all quarry state cleared on `EndSession`

The specification can be type-checked with `fuzz -t claude-code-quarry.tex`.

# A Operation Concurrency Model

Every quarry operation is exposed on up to three surfaces: the HTTP API (daemon), the CLI, and the MCP server (stdio or WebSocket). The concurrency model differs by surface because each has different constraints: HTTP connections time out, MCP tool calls block the LLM response stream, and CLI commands run in a user's terminal.

Table 4 classifies each operation. The **Client model** column describes what the caller experiences:

**sync** The caller blocks until the operation completes and receives the result inline.

**fire-and-forget** The caller receives an acknowledgement immediately (HTTP 202 or a short string). Work runs in the background; the caller polls or ignores the outcome.

— The operation is not available on that surface.

## A.1 Concurrency Matrix

Table 4: Operation concurrency by surface. *Long-running* = involves network I/O, embedding, or bulk LanceDB writes.

| Operation | HTTP API | CLI (local) | CLI (remote) | MCP |
|---|---|---|---|---|
| *Read operations* | | | | |
| find (search) | sync | sync | sync | sync |
| show | sync | sync | sync | sync |
| status | sync | sync | sync | sync |
| list documents | sync | sync | sync | sync |
| list collections | sync | sync | sync | sync |
| list registrations | sync | sync | sync | sync |
| list databases | sync | sync | sync | sync |
| health | sync | — | — | — |
| *Mutating operations* | | | | |
| register | fire-and-forget | sync | fire-and-forget | fire-and-forget |
| deregister | fire-and-forget | sync | fire-and-forget | fire-and-forget |
| delete (document) | fire-and-forget | sync | fire-and-forget | fire-and-forget |
| delete (collection) | fire-and-forget | sync | fire-and-forget | fire-and-forget |
| remember | fire-and-forget | sync | fire-and-forget | fire-and-forget |
| ingest (URL) | fire-and-forget | sync | fire-and-forget | fire-and-forget |
| ingest (file) | — | sync | — | fire-and-forget |
| sync | fire-and-forget | sync | fire-and-forget | fire-and-forget |
| use (switch db) | sync | sync | sync | sync |
| optimize | — | sync | — | — |

## A.2 Design Rationale

**Read operations** are synchronous on all surfaces. Hybrid search (embedding + vector similarity + BM25) completes in <100 ms after model load; the overhead of task tracking would exceed the operation itself.

**Mutating operations** are all fire-and-forget on HTTP. Every mutating endpoint returns 202 Accepted with a `task_id`; work runs in a background `asyncio.Task`. This eliminates the 15-second CLI timeout as a failure mode for any write operation.

- All mutating endpoints use `GET /tasks/{task_id}` for status polling. `/sync/{id}` and `/ingest/{id}` are backwards-compatible aliases.
- **Sync** rejects concurrent requests with HTTP 409 because a second sync against the same registrations would race on LanceDB writes. All other operations allow concurrency.

- Completed and failed tasks are garbage-collected after a 1-hour TTL on the next task creation.
- **Optimize** is CLI-only (no HTTP or MCP surface). It runs LanceDB compaction and index rebuilds, which are CPU-intensive and should not be triggered remotely without explicit intent.

## A.3 Task Lifecycle

All background tasks follow the same lifecycle:

1. `POST` validates input, creates a `TaskState` record on the server context, spawns an `asyncio.Task` wrapping `run_in_threadpool()`, and returns HTTP 202 with `task_id` and `status` = `accepted`.
2. The background task updates `status` to `completed` (with `results`) or `failed` (with `error`) on completion.
3. A `finally` guard marks any task that exits without setting a terminal status as `failed`.
4. `CancelledError` is caught before `Exception` to ensure clean shutdown on server stop.
5. The CLI prints the `task_id` and exits 0 (fire-and-forget).